

Vincenzo Rubulotta
Matricola 1000034149

Relazione Homework 1 Prova in Itinere

DSBD

1. Introduzione

In questo documento verranno esposte le scelte implementative architetturali del progetto sviluppato nel contesto del corso di Sistemi Distribuiti e Big Data. L'obiettivo dell'elaborato è la progettazione e lo sviluppo di un'architettura a microservizi containerizzata, in grado di gestire la registrazione di utenti e il monitoraggio, sia ciclico che su richiesta diretta, dei dati di traffico aereo forniti dalla piattaforma OpenSky Network.

In conformità alle specifiche di progetto, il sistema è stato decomposto in quattro componenti principali, ciascuno incapsulato in un proprio container Docker. L'architettura si articola nei seguenti moduli: un User Manager per la gestione delle identità, un Data Collector per l'interazione con le sorgenti dati esterne e due database distinti, basati su istanze PostgreSQL isolate, dedicati a ciascun microservizio. Infine, è stato introdotto un API Gateway che funge da punto di ingresso unico per gestire le richieste HTTP REST provenienti dai client esterni e instradarle verso la logica interna.

Particolare attenzione è stata posta nella definizione dei protocolli di comunicazione: il sistema come già detto in precedenza espone un'interfaccia standard verso il client, ma adotta il protocollo gRPC basato su HTTP/2 e Protobuf per tutte le comunicazioni inter-servizio, garantendo bassa latenza. La relazione descriverà inoltre le strategie adottate per garantire la robustezza del sistema, come l'isolamento delle reti Docker, la gestione della politica "At-Most-Once".

2. Schema Architetture

Nel seguente schema a blocchi è illustrata la logica del sistema preso in esame:

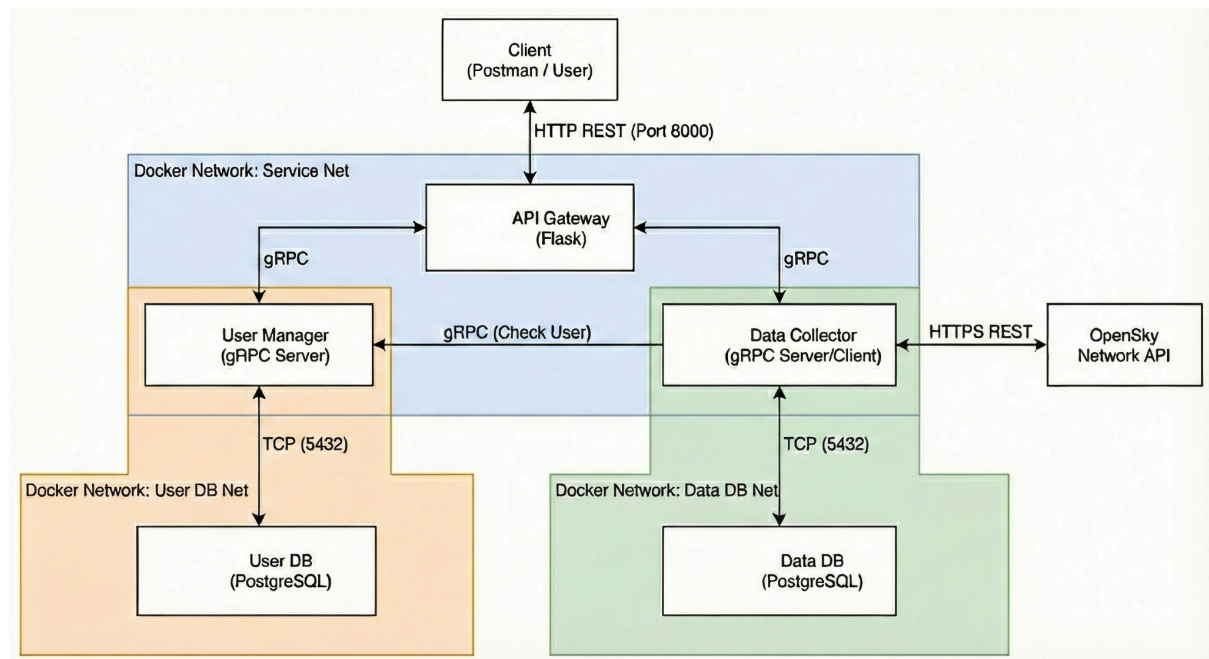


Figura 1 - Diagramma architetturale dei componenti e flussi di comunicazione

2.1 Descrizione dei componenti

Di seguito si procederà con la descrizione dei componenti principali del sistema, ognuno dei quali isolato nel proprio container per garantire modularità.

API Gateway

Realizzato su Python (flask), questo rappresenta l'unico punto di accesso al sistema per il client esterno. Svolge un compito di disaccoppiamento dell'interfaccia pubblica del sistema, con la logica interna del sistema stesso, questo viene effettuato raccogliendo le richieste HTTP in formato JSON, che vengono ricevute dall'esterno, nel nostro caso specifico da POSTMAN, in chiamate gRPC che vengono indirizzate verso i due microservizi.

User Manager

Questo è uno dei due microservizi ed è responsabile della gestione dei dati relativi ai vari utenti registrati al sistema. Questo espone tramite interfaccia gRPC un metodo per la registrazione (RegisterUser), un metodo per la cancellazione (DeleteUser) e un terzo metodo per la verifica dell'esistenza di un determinato utente (CheckUserExists). Inoltre questo è l'unico punto di accesso al User DB.

Data Collector

Il secondo microservizio è responsabile della raccolta dei dati relativi ai vari voli. Questo componente svolge diverse funzioni principali: si comporta da server tramite l'interfaccia gRPC che espone tre metodi, uno per la registrazione degli interessi di un utente su un determinato aeroporto (*RegisterInterest*), il secondo per raccogliere i dati sul più recente volo in ingresso/uscita da un determinato aeroporto (*GetHistoricalData*) e un metodo per ottenere la media dei voli di un determinato aeroporto in un pool di giorni scelto dall'utente (*GetStatistics*).

Questo componente si comporta anche da client gRPC, in quanto interroga lo User Manager per validare alcune richieste, come la richiesta di registrazione degli interessi; infine, ha un terzo comportamento da worker ciclico, eseguendo un thread in background per interrogare ciclicamente le API OpenSky Network in base agli interessi registrati dai vari utenti.

Layer di Persistenza

Il layer di persistenza è composto da due DB separati, gestiti da due istanze separate di PostgreSQL: User DB e Data DB. Il primo si occupa della registrazione dei dati relativi a ogni utente ed è accessibile solo tramite lo User Manager. Il secondo, analogamente, ha come unico punto di accesso il Data Collector e raccoglie tutte le informazioni relative ai vari voli, ottenute dal monitoraggio ciclico o tramite il metodo *GetHistoricalData* richiesto dall'utente; inoltre, contiene le informazioni relative agli interessi registrati da ogni utente.

2.2 Topologia delle Network

Una delle caratteristiche principali dell'architettura è l'isolamento di rete, realizzato tramite l'uso di Docker Networks personalizzate. Invece di utilizzare un'unica rete condivisa, il sistema è diviso in tre segmenti distinti.

La comunicazione tra i servizi principali, ovvero API Gateway, User Manager e Data Collector, avviene sulla *service_net*, una rete di tipo bridge dove lo scambio di dati utilizza il protocollo gRPC. Per la gestione dei dati, invece, sono state create due reti private e isolate: la *user_db_net*, che collega esclusivamente lo User Manager al suo database, e la *data_db_net*, che connette il Data Collector al proprio database. Questa separazione garantisce che non ci sia alcun collegamento diretto, nemmeno a basso livello, tra il Data Collector e il database degli utenti, aumentando notevolmente la sicurezza dell'intero sistema.

3. Scelte Progettuali e Motivazioni

In questa sezione vengono analizzate le decisioni architetturali critiche adottate durante lo sviluppo del sistema, motivandole nel contesto dei sistemi distribuiti e delle specifiche assegnate.

3.1 Adozione del API Gateway

Per la gestione degli accessi è stato introdotto un microservizio dedicato che agisce da API Gateway, sviluppato con Flask, che funge da unico punto di ingresso per le chiamate esterne. Sebbene fosse possibile esporre direttamente le porte dei singoli microservizi, l'adozione di questo pattern offre vantaggi decisivi.

In primo luogo, consente di separare i protocolli di comunicazione: il traffico interno rimane su gRPC, garantendo efficienza e tipizzazione, mentre all'esterno viene esposta un'interfaccia REST standard, facilmente utilizzabile da client come Postman. Inoltre, il Gateway centralizza la gestione del routing e nasconde la complessità dell'architettura interna; in questo modo, il client interagisce con un solo endpoint sulla porta 8000, senza la necessità di conoscere gli indirizzi specifici di tutti i microservizi.

3.2 Isolamento delle reti Docker

L'organizzazione dell'infrastruttura in tre reti distinte (`service_net`, `user_db_net` e `data_db_net`) gioca un ruolo cruciale nel garantire un effettivo disaccoppiamento tra i componenti del sistema. Isolando i database all'interno di reti private dedicate, si forza una separazione netta delle responsabilità: nessun microservizio ha la possibilità tecnica di accedere direttamente ai dati gestiti da un altro componente.

Questa configurazione obbliga i servizi a interagire esclusivamente attraverso le interfacce ben definite esposte sulla `service_net`, eliminando il rischio di dipendenze nascoste a livello di database. Di conseguenza, ogni microservizio opera come un'unità autonoma che nasconde la propria complessità interna, migliorando non solo la sicurezza, ma anche la modularità e la manutenibilità dell'intera architettura.

3.3 Adozione di PostgreSQL

Per la gestione della persistenza dei dati si è deciso di adottare PostgreSQL, configurando due istanze distinte dedicate rispettivamente al gestore degli utenti e al collettore dei dati. La decisione di utilizzare un sistema di gestione di database relazionali, preferendolo a soluzioni non relazionali o basate sulla memoria principale, nasce dalla natura fortemente strutturata delle informazioni trattate. Tali dati richiedono uno schema rigido e la necessità di garantire l'integrità dei riferimenti e le proprietà fondamentali delle transazioni, ovvero atomicità, coerenza, isolamento e durabilità.

Infine, l'uso di istanze separate implementa il modello architetturale che prevede un database dedicato per ogni servizio, rafforzando la separazione tra i componenti e impedendo la creazione di dipendenze dirette sui dati.

3.4 Separazione delle Definizioni Protobuf

Per la definizione delle interfacce si è deciso di utilizzare due file distinti e semanticamente separati, denominati `server.proto` e `data_collector.proto`, evitando così l'uso di un unico file monolitico condiviso. Tale scelta garantisce una maggiore modularità, in quanto rende ogni servizio proprietario della propria interfaccia; di conseguenza, le modifiche apportate allo schema del Data Collector non impongono la ricompilazione o l'aggiornamento dello User Manager, favorendo l'evoluzione indipendente dei microservizi.

Inoltre, questa netta separazione ha risolto alla radice i conflitti relativi agli spazi dei nomi nel Descriptor Pool di Protocol Buffers, problematiche che emergevano tentando di caricare definizioni duplicate all'interno del medesimo ambiente di esecuzione Python, come accadeva ad esempio nel Gateway che necessita di importare entrambi i moduli.

3.5 Politica "At-Most-Once" tramite Request ID

Per garantire l'idempotenza nelle operazioni di registrazione si è scelto di associare a ogni richiesta un identificativo univoco universale `request_id`. Questa decisione nasce dalla consapevolezza che, in un sistema distribuito, affidarsi esclusivamente ai vincoli del database, come l'unicità dell'email, non è sufficiente; tale approccio, infatti, non permette di distinguere un errore reale, causato da dati non validi, da un nuovo tentativo automatico generato da instabilità della rete.

A livello implementativo, il sistema traccia ogni identificativo già elaborato in una tabella dedicata denominata `ProcessedRequests`. Se il server riceve una richiesta con un identificativo già presente, riconosce che si tratta di un duplicato, dovuto ad esempio a una ritrasmissione per scadenza del tempo di attesa, e restituisce immediatamente l'esito positivo dell'operazione precedente senza rieseguirlo. Questo meccanismo assicura in modo robusto che l'elaborazione avvenga una e una sola volta, rispettando la semantica *Exactly-Once*.

4. API e Flussi di Comunicazione

In questa sezione vengono descritti in dettaglio gli endpoint esposti dall'API Gateway. Per ogni API, viene illustrato il metodo di accesso, i dati richiesti e il flusso di comunicazione interno tra i microservizi coinvolti.

4.1 Registrazione Utente

La funzionalità di registrazione permette la creazione di un nuovo profilo utente nel sistema, garantendo che l'operazione venga eseguita al massimo una volta grazie ai vincoli di integrità definiti nel database. Il flusso di comunicazione ha inizio quando il client invia i dati anagrafici in formato JSON all'API Gateway tramite l'interfaccia HTTP REST.

Una volta ricevuta la richiesta, il Gateway si occupa di tradurla nel messaggio Protocol Buffers denominato UserData e invoca il servizio User Manager utilizzando il protocollo gRPC. Quest'ultimo tenta l'inserimento delle informazioni nel proprio database PostgreSQL e, in caso di esito positivo, restituisce il codice di stato 201 Created al Gateway, il quale provvede infine a inoltrare la conferma al client.

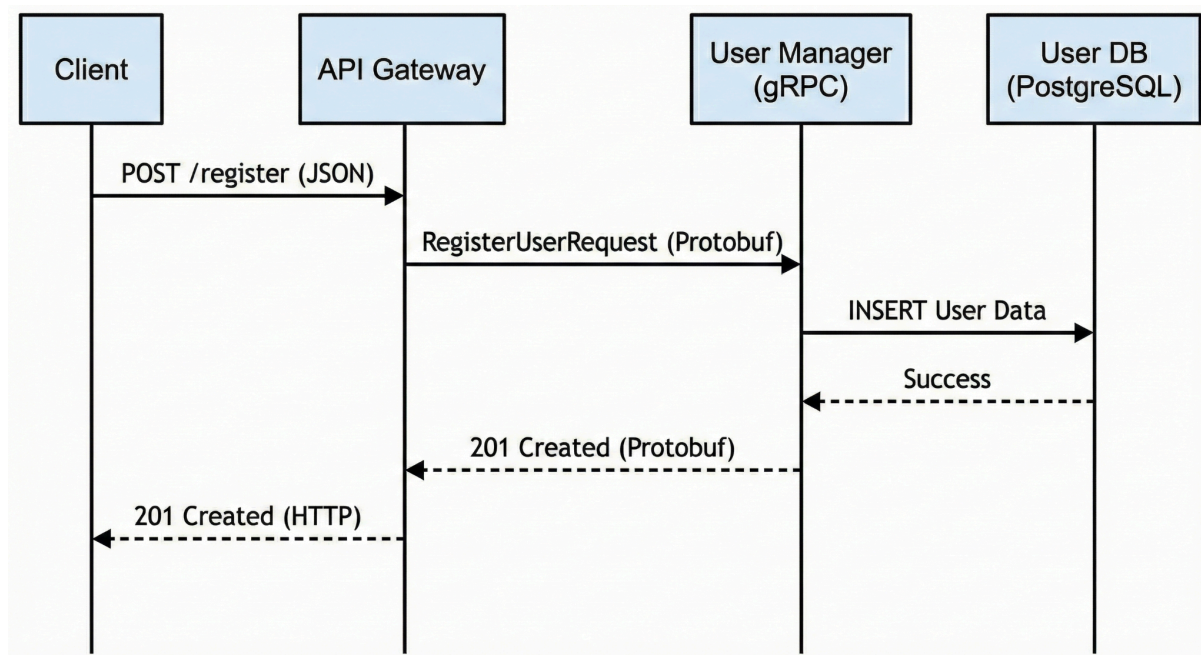


Figura 2: Diagramma di sequenza Registrazione Utente

4.2 Cancellazione Utente

La funzionalità di eliminazione consente la rimozione definitiva di un utente dal sistema, garantendo che l'operazione sia idempotente e che, al termine, i dati non siano più accessibili. Il flusso di comunicazione ha inizio quando il client invia una richiesta HTTP con metodo DELETE all'API Gateway, specificando l'indirizzo email dell'utente all'interno dell'URL.

Ricevuta la richiesta, il Gateway estrae l'email per costruire il messaggio gRPC UserIdentifier e invoca la procedura remota DeleteUser sul microservizio User Manager. Quest'ultimo esegue la query di cancellazione sul proprio database PostgreSQL isolato e, in base alla risposta del database, comunica l'esito al Gateway; infine, il risultato viene tradotto in un codice di stato HTTP per il client, restituendo un 200 OK se la cancellazione è avvenuta con successo o un 404 Not Found qualora l'utente non fosse presente nel sistema.

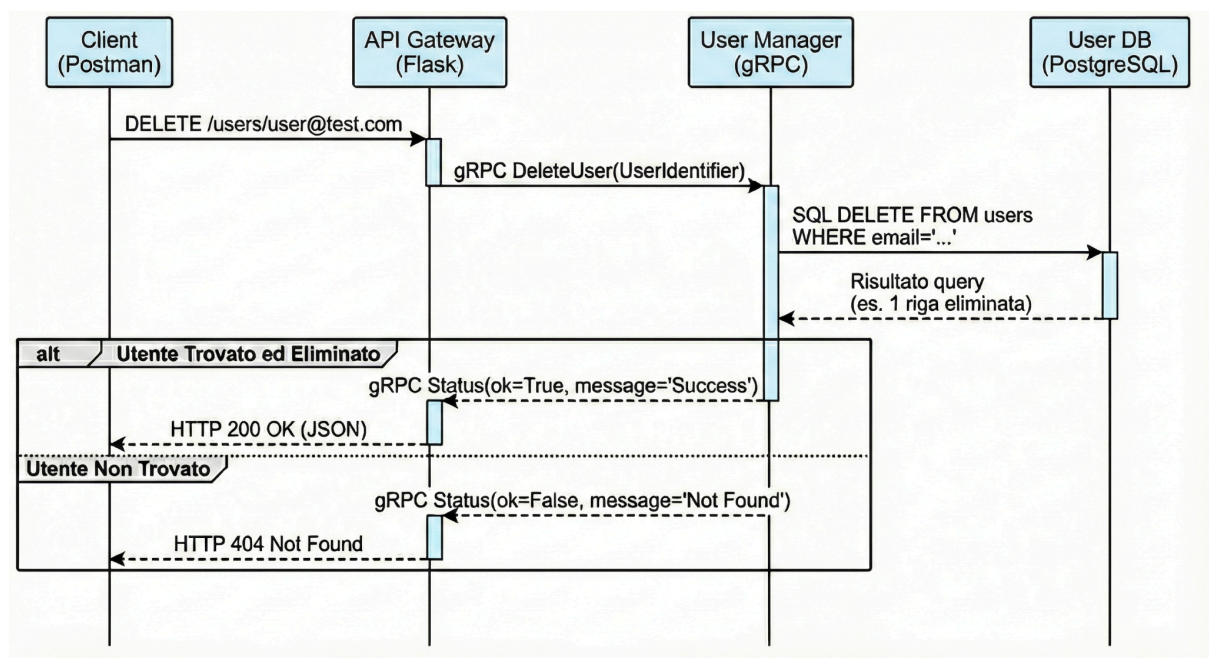


Figura 3: Diagramma di sequenza Eliminazione Utente

4.3 Verifica Esistenza Utente

La funzionalità di verifica consente di accertare la presenza di un determinato utente all'interno del sistema. Si tratta di un'operazione di sola lettura, spesso utilizzata in fase preliminare rispetto ad altre attività, come la registrazione di nuovi interessi, per prevenire eventuali errori operativi.

Il flusso di comunicazione si attiva quando il client invia una richiesta HTTP di tipo GET all'API Gateway, specificando l'indirizzo email da verificare nel percorso dell'URL. A quel punto, il Gateway invoca la procedura remota CheckUserExists sul microservizio User Manager tramite gRPC, il quale esegue una query di selezione sul proprio database per cercare la corrispondenza con la chiave primaria. Una volta ottenuto l'esito, lo User Manager lo incapsula in un messaggio booleano e lo restituisce al Gateway, che provvede infine a inoltrare la risposta in formato JSON al client con codice di stato 200.

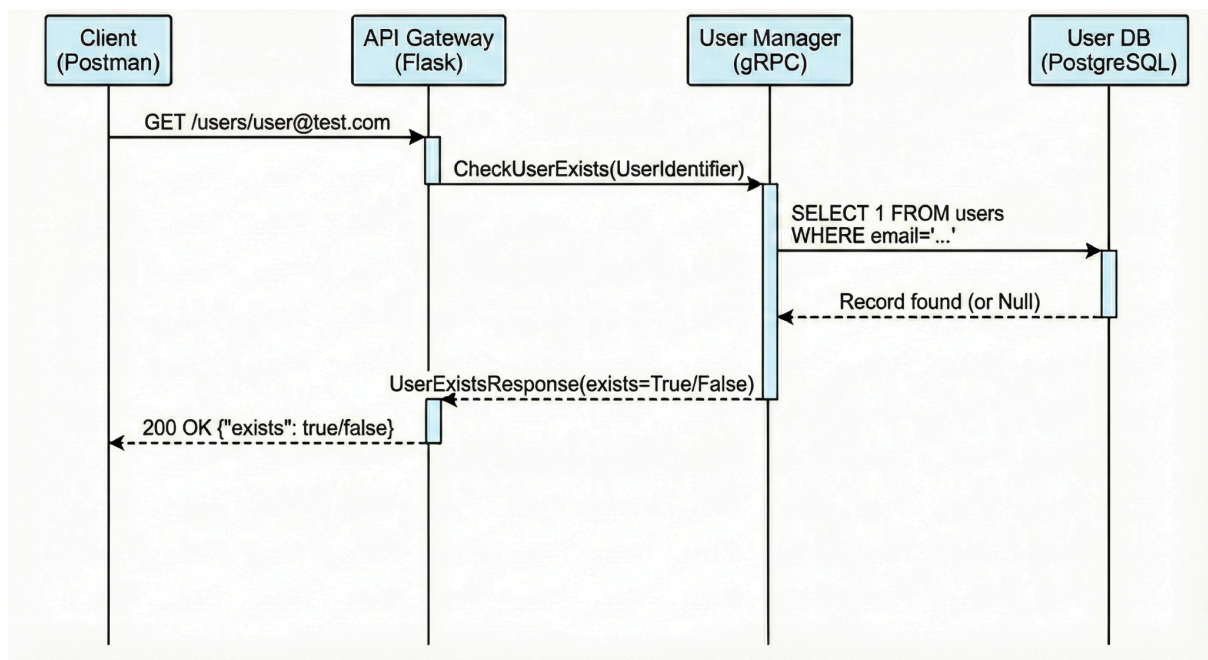


Figura 4: Diagramma di sequenza Verifica Esistenza Utente

4.4 Registrazione Interessi

La funzionalità di registrazione degli interessi consente all'utente di definire o aggiornare la lista degli aeroporti di proprio gradimento, abilitando di conseguenza il Data Collector al monitoraggio ciclico dei relativi voli. Il processo di comunicazione prende avvio quando il client invia una richiesta HTTP di tipo POST all'API Gateway, contenente l'indirizzo email dell'utente e l'elenco dei codici ICAO degli aeroporti desiderati.

Successivamente, il Gateway inoltra la richiesta al Data Collector invocando la procedura gRPC RegisterInterest. A questo punto, prima di procedere con il salvataggio dei dati, il Data Collector agisce a sua volta come client interrogando lo User Manager tramite la procedura CheckUserExists, al fine di verificare la validità dell'utente. Solo in caso di riscontro positivo, la nuova lista di interessi viene salvata o sovrascritta nel database del Data Collector, il quale conferma l'operazione al Gateway che provvede infine a restituire al client il codice di stato 200 OK.

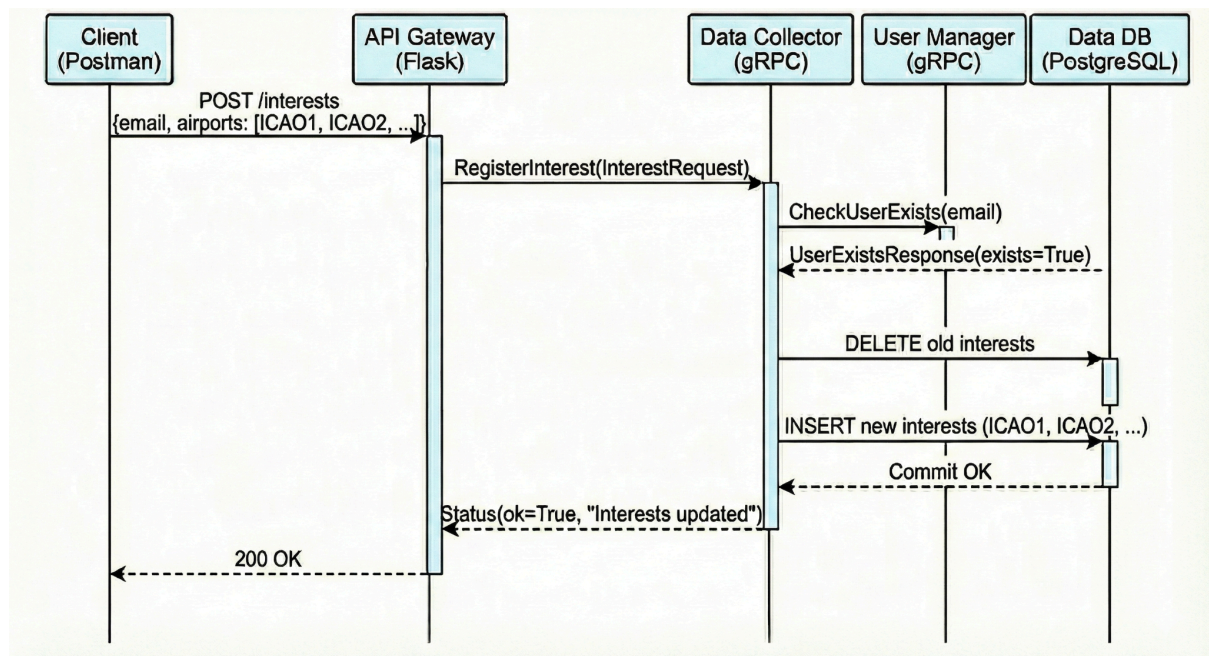


Figura 5: Diagramma di sequenza Registrazione Interessi

4.5 Recupero Dati Volo

La funzionalità dedicata al recupero dei dati storici consente di ottenere le informazioni relative agli arrivi e alle partenze per un determinato aeroporto, implementando una strategia di caching intelligente volta a garantire bassa latenza. Il sistema verifica prioritariamente la presenza dei dati nel database locale (Data DB); qualora questi siano assenti o obsoleti, configurando uno scenario di Cache Miss, il Data Collector interroga in tempo reale l'API esterna OpenSky Network, salvando i risultati per future richieste prima di restituirli all'utente.

Il flusso di comunicazione ha inizio quando il client invia una richiesta HTTP di tipo GET contenente l'email dell'utente e il codice dell'aeroporto. Ricevuta la chiamata, l'API Gateway invoca la procedura gRPC GetHistoricalData sul Data Collector, il quale esegue innanzitutto una verifica preliminare dell'utente contattando lo User Manager. Successivamente, viene eseguita l'interrogazione sul database: in caso di Cache Hit, i dati vengono restituiti immediatamente, mentre in caso contrario il servizio recupera le nuove informazioni da OpenSky, le memorizza nel database e infine inoltra la risposta al Gateway.

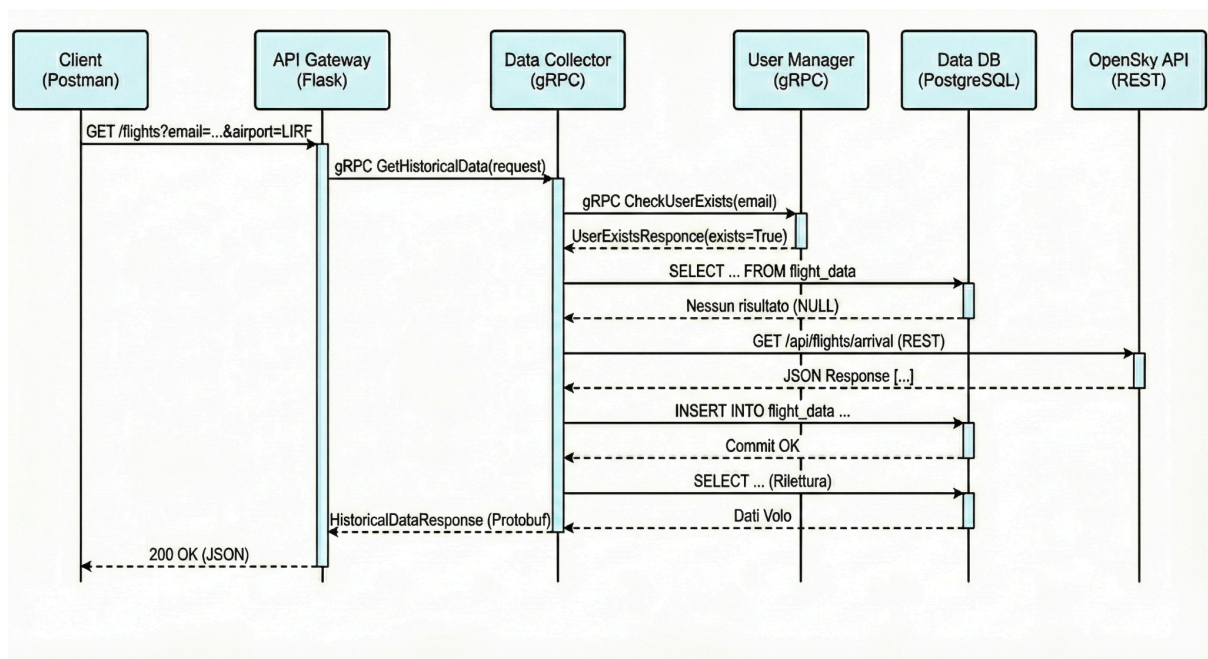


Figura 6: Diagramma di sequenza Recupero Dati Volo

4.6 Statistiche Voli

La funzionalità dedicata alle statistiche fornisce analisi aggregate sui dati di volo storicizzati, calcolando nello specifico la media giornaliera di arrivi e partenze per un determinato aeroporto in un intervallo temporale che, se non specificato dall'utente, copre di default gli ultimi sette giorni. È importante sottolineare che questa operazione viene eseguita esclusivamente attingendo alle informazioni già presenti nel database locale Data DB, senza la necessità di interrogare API esterne in tempo reale.

Il flusso di comunicazione si attiva quando il client invia una richiesta HTTP di tipo GET includendo i parametri necessari, ovvero l'email per l'autorizzazione, il codice ICAO dell'aeroporto e, opzionalmente, la finestra temporale di interesse. L'API Gateway inoltra la richiesta invocando la procedura gRPC GetStatistics sul Data Collector, il quale provvede preliminarmente a verificare l'effettiva registrazione dell'utente contattando lo User Manager. Una volta confermata la validità del richiedente, il servizio calcola il periodo di riferimento ed esegue le query di aggregazione sul proprio database per conteggiare i voli; infine, elabora le medie matematiche e trasmette i risultati tramite messaggio Protobuf al Gateway, che restituisce al client un oggetto JSON contenente i valori calcolati.

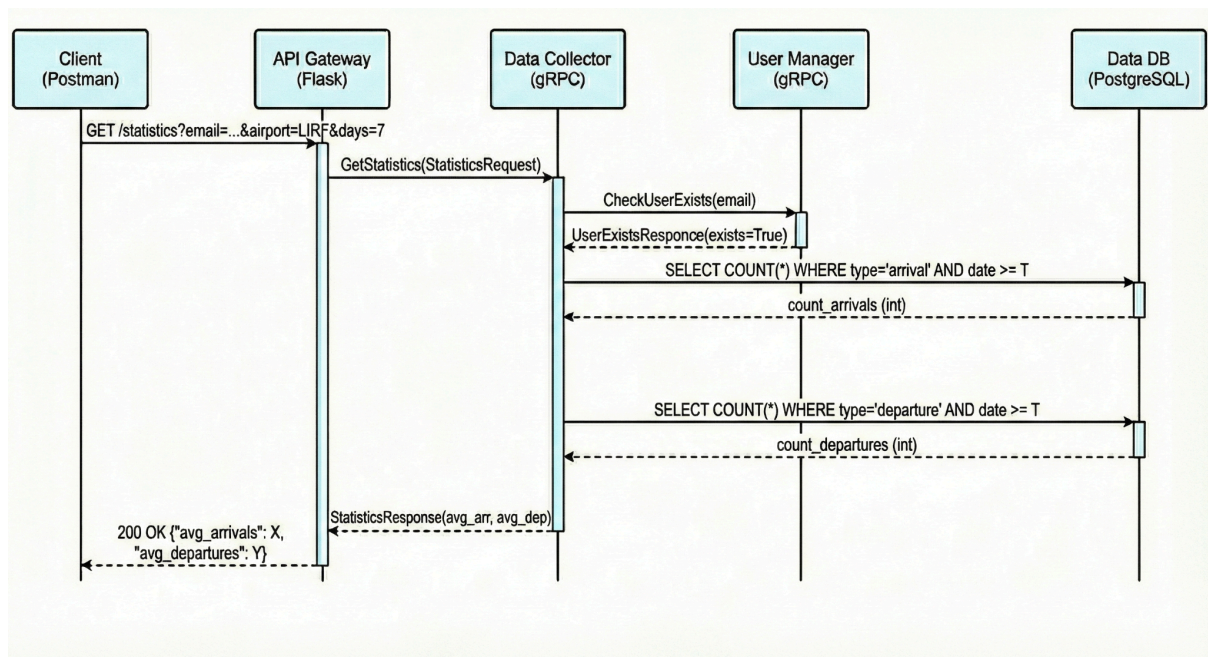


Figura 7: Diagramma di sequenza Recupero Statistiche Voli